

RevSecureCloud — Knowledge Transfer (Functional)

Version: 3.0.0 Date: 2026-03-07

Purpose

This functional KT is targeted at **product owners, QA engineers, and frontend engineers**. It describes user journeys, the full API surface, role permissions, AI features, enterprise governance behaviours (v3.0), and acceptance criteria for all implemented phases.

Product Overview

RevSecureCloud is a multi-tenant B2B SaaS platform that helps subscription businesses detect, analyse, and recover lost or at-risk revenue. The system:

- Connects to external revenue providers (Stripe, GitHub, custom HTTP APIs)
- Ingests and normalises revenue records
- Detects revenue leakages across 7 categories
- Runs AI-powered analysis to surface anomalies, forecasts, and risk scores
- Proposes automated recovery actions via an AI Agent (human-in-loop approval required)
- Maintains a full audit trail of every action

Phase Delivery Summary

Phase	Feature	Status
1	Multi-tenant auth, revenue records, leakage detection, audit trail	✓ Done
2	Integrations (connector v1), billing, API keys, provider connections	✓ Done
3	AI Intelligence Engine (anomaly detection, forecasting, risk scoring, LLM summaries)	✓ Done
4	AI Agent (action proposals, human approval, execution, audit by proposal)	✓ Done
4+	Enterprise Governance (request tracing, idempotency, rate limits, AI policy, double-approval, DLQ, AI safety, job observability, test suite)	✓ Done

User Roles & Permissions

Role	Description	Key Access
SUPER_ADMIN	Platform operator	All tenants, all endpoints
COMPANY_ADMIN	Tenant administrator	All company endpoints including AI, agent, audit
ANALYST	Revenue analyst	Revenue, leakages, AI insights (read)

VIEWER	Read-only stakeholder	Revenue, leakages (read-only)
--------	-----------------------	-------------------------------

Primary User Journeys

1. Authentication

Sign in:

```
POST /api/v1/auth/login
body: { "email": "admin@demo.com", "password": "password123" }
```

Returns: { user, accessToken, refreshToken } . Emits a LOGIN audit event.

Refresh tokens:

```
POST /api/v1/auth/refresh
body: { "refreshToken": "..." }
```

Demo credentials (local + production Neon):

- Email: admin@demo.com | Password: password123 | Role: COMPANY_ADMIN

2. Revenue Management

- List revenue records with optional filters (period , product , customer , currency): GET /api/v1/revenue
- Create a revenue record manually: POST /api/v1/revenue
- Delete a record (emits DELETE_REVENUE audit event): DELETE /api/v1/revenue/:revenueId

3. Revenue Leakage Investigation

Leakages are pre-computed financial losses grouped by category. An analyst can:

- List leakages — filterable by category , isResolved , riskScore range: GET /api/v1/revenue/leakages
- Resolve a leakage — marks it resolved and emits RESOLVE_LEAKAGE audit event: PATCH /api/v1/revenue/leakages/:leakageId/resolve

Leakage Categories: CHURN , DUNNING_FAILURE , PRICING_GAP , FAILED_UPSELL , REFUND , DISCOUNT_ABUSE , INVOICE_ERROR

4. Provider Integrations (Connectors)

Admins connect external providers to pull live revenue data:

- List available integration providers: GET /api/v1/integrations
- Connect a provider (Stripe, GitHub, or custom): POST /api/v1/integrations
- List custom providers (user-defined HTTP APIs): GET /api/v1/integrations/custom-providers
- Create custom provider: POST /api/v1/integrations/custom-providers
- List built-in icon presets for the icon picker: GET /api/v1/integrations/icons/presets
- Get test credentials (dev/staging pre-filled credentials for Connect modal): GET /api/v1/integrations/test-credentials/:providerId

5. AI Intelligence Engine (Phase 3)

The AI engine analyses revenue data and produces structured insights with optional LLM-generated narratives.

Running an AI Analysis

```
POST /api/v1/ai/run
Headers: Authorization: Bearer <token> (COMPANY_ADMIN)
body: {
  "modelId": "groq/llama-3.1-8b-instant", // optional – uses default if omitted
  "enrichWithLLM": true                // optional – triggers LLM summary
}
```

The engine runs three sub-engines in parallel:

- 1. **Anomaly Detector** — Z-score analysis for revenue spikes and drops
- 2. **Forecast Engine** — 30-day trend projection with confidence interval
- 3. **Risk Scoring Engine** — composite 0–100 score weighing churn, dunning, leakage density

Returns a saved `AIInsight` record with `severity` (LOW/MEDIUM/HIGH/CRITICAL), `riskScore`, `title`, and `summary`.

Reading Insights

- **List all insights** (paginated, newest first): `GET /api/v1/ai/insights?page=1&limit=20`
- **Get single insight** (full data payload): `GET /api/v1/ai/insights/:insightId`
- **Mark insight as read**: `PATCH /api/v1/ai/insights/:insightId/read`
- **List available AI models**: `GET /api/v1/ai/models`
- **Get AI usage quota**: `GET /api/v1/ai/usage`

Insight Severity Thresholds

Severity	Risk Score	Meaning
LOW	0–24	Watch — no immediate action required
MEDIUM	25–49	Monitor — review leakage categories
HIGH	50–74	Act soon — significant revenue at risk
CRITICAL	75–100	Urgent — major revenue loss or anomaly detected

6. AI Agent — Action Proposals (Phase 4)

The AI Agent reads an insight and generates structured action proposals. **All proposals require human approval before execution** (human-in-loop).

Generate Proposals for an Insight

```
POST /api/v1/agent/plan
Headers: Authorization: Bearer <token> (COMPANY_ADMIN)
body: { "insightId": "cmmXXX..." }
```

Or from the standalone agent dashboard (no specific insight):

```
{ "context": "Analyze all revenue data and identify recovery actions" }
```

Response:

```
{
  "insightId": "cmmXXX...",
  "proposals": [ ... ],
  "totalCreated": 5,
  "existingCount": 0
}
```

- `totalCreated` — new proposals created in this call
- `existingCount` — existing PROPOSED proposals already on record (dedup prevented re-creation)

Deduplication: The backend uses **payload-aware deduplication** — proposals are only skipped if they have the exact same `actionType` AND the same payload. Multiple proposals of the same type (e.g. `RETRY_PAYMENT`) targeting different customer segments are all allowed.

Action Types

Action Type	Description
RETRY_PAYMENT	Retry failed payment charges
SEND_COLLECTION_EMAIL	Send overdue payment reminders
FLAG_HIGH_RISK_ACCOUNT	Flag account for risk review
GENERATE_INVOICE	Generate missing or corrected invoice
DISABLE_SUBSCRIPTION	Suspend delinquent subscription
NOTIFY_FINANCE_TEAM	Alert the finance team

Proposal Lifecycle

```
PROPOSED → APPROVED → EXECUTED
      ↓
      REJECTED
```

- **List proposals** (paginated, filterable by `status`, `actionType`): `GET /api/v1/agent/proposals`
- **Approve a proposal:** `POST /api/v1/agent/proposals/:id/approve`
- **Reject a proposal:** `POST /api/v1/agent/proposals/:id/reject`
- **Execute an approved proposal** (requires `estimatedImpact` ≥ 100 and `confidenceScore` ≥ 0.6):
`POST /api/v1/agent/proposals/:id/execute` ← body must include `{}`
- **View execution history:** `GET /api/v1/agent/proposals/:id/execution-logs`
- **Remove duplicate PROPOSED proposals (admin utility):** `POST /api/v1/agent/proposals/cleanup` ← body must include `{}`

7. Audit Trail

Every significant action is recorded in the audit trail (immutable, append-only).

- **List audit events** (COMPANY_ADMIN only): GET /api/v1/audit?page=1&limit=50&action=LOGIN&search=john
- Filters: action (exact match), search (searches actorEmail, actorName, detail)
- Each record includes: actorEmail , actorName , action , resourceType , resourceId , before (JSON snapshot), after (JSON snapshot), ipAddress , userAgent , createdAt

Audit events currently emitted:

Event	Trigger
LOGIN	Successful login
DELETE_REVENUE	Revenue record deleted
RESOLVE_LEAKAGE	Leakage marked as resolved
AGENT_PROPOSAL_APPROVED	Proposal approved
AGENT_PROPOSAL_REJECTED	Proposal rejected
AGENT_PROPOSAL_EXECUTED	Proposal executed

8. Billing & Subscription

- GET /api/v1/billing/plan — current subscription plan and status
- POST /api/v1/billing/webhook — Stripe webhook receiver (handles invoice.paid , customer.subscription.*)
- Plan tiers: STARTER , GROWTH , ENTERPRISE

9. User & Company Management

- GET /api/v1/users/me — current user profile
- PATCH /api/v1/users/me — update profile
- GET /api/v1/companies/me — current company details
- PATCH /api/v1/companies/me — update company settings

Full API Route Table

Method	Path	Min Role	Description
POST	/api/v1/auth/login	Public	Login
POST	/api/v1/auth/refresh	Public	Refresh tokens
GET	/api/v1/revenue	VIEWER	List revenue records
POST	/api/v1/revenue	COMPANY_ADMIN	Create revenue record
DELETE	/api/v1/revenue/:id	COMPANY_ADMIN	Delete revenue record
GET	/api/v1/revenue/leakages	ANALYST	List leakages
POST	/api/v1/revenue/leakages	COMPANY_ADMIN	Create leakage

PATCH	/api/v1/revenue/leakages/:id/resolve	COMPANY_ADMIN	Resolve leakage
GET	/api/v1/audit	COMPANY_ADMIN	Audit trail
GET	/api/v1/integrations	ANALYST	List integrations
POST	/api/v1/integrations	COMPANY_ADMIN	Connect provider
GET	/api/v1/integrations/custom-providers	ANALYST	List custom providers
POST	/api/v1/integrations/custom-providers	COMPANY_ADMIN	Create custom provider
GET	/api/v1/integrations/icons/presets	Any authed	Icon presets
GET	/api/v1/integrations/test-credentials/:id	Any authed	Test credentials
POST	/api/v1/ai/run	COMPANY_ADMIN	Run AI analysis
GET	/api/v1/ai/insights	ANALYST	List AI insights
GET	/api/v1/ai/insights/:id	ANALYST	Get insight detail
PATCH	/api/v1/ai/insights/:id/read	ANALYST	Mark insight as read
GET	/api/v1/ai/models	Any authed	List AI models
GET	/api/v1/ai/usage	COMPANY_ADMIN	AI usage quota
POST	/api/v1/agent/plan	COMPANY_ADMIN	Generate action proposals
GET	/api/v1/agent/proposals	Any authed	List proposals
POST	/api/v1/agent/proposals/:id/approve	COMPANY_ADMIN	Approve proposal
POST	/api/v1/agent/proposals/:id/reject	COMPANY_ADMIN	Reject proposal
POST	/api/v1/agent/proposals/:id/execute	COMPANY_ADMIN	Execute proposal
GET	/api/v1/agent/proposals/:id/execution-logs	Any authed	Execution history
POST	/api/v1/agent/proposals/cleanup	COMPANY_ADMIN	Deduplicate proposals
GET	/api/v1/billing/plan	COMPANY_ADMIN	Billing info
POST	/api/v1/billing/webhook	Public (Stripe sig)	Stripe webhook
GET	/api/v1/users/me	Any authed	Own profile
PATCH	/api/v1/users/me	Any authed	Update profile
GET	/api/v1/companies/me	Any authed	Company details
PATCH	/api/v1/companies/me	COMPANY_ADMIN	Update company
GET	/health	Public	Health check

Note: All POST bodies must include a JSON body (at minimum `{}`). The global raw-body parser (Stripe webhook) rejects bodyless POST requests with a `500`.

Response Envelope

All API responses now include `requestId` and `timestamp` (v3.0+):

```
{
  "success": true,
  "data": { ... },
  "message": "Created successfully",
  "requestId": "b3d1a2f4-8e7c-4b1a-9f2d-0c3e5a6b7d8e",
  "timestamp": "2026-03-07T12:00:00.000Z"
}
```

Error responses:

```
{
  "success": false,
  "error": "Human readable message",
  "code": "MACHINE_CODE",
  "requestId": "b3d1a2f4-8e7c-4b1a-9f2d-0c3e5a6b7d8e",
  "timestamp": "2026-03-07T12:00:00.000Z"
}
```

Rate limit error (HTTP 429):

```
{
  "success": false,
  "error": "Rate limit exceeded",
  "code": "RATE_LIMITED",
  "retryAfter": 30,
  "requestId": "...",
  "timestamp": "..."
}
```

Frontend guidance: Always capture `requestId` from error responses and include it in support tickets / logs. The `X-Request-Id` response header carries the same value.

Enterprise Governance — Functional Behaviour (NEW v3.0)

1. Request Tracing

Every HTTP response carries an `X-Request-Id` header containing a UUID-v4. The same value appears as `requestId` in the response JSON body.

- Frontend should log `requestId` on every API error for support correlation.
- Backend logs all include the same `requestId` so engineers can trace a request end-to-end.

2. Rate Limiting by Plan

Requests are rate-limited per company (`companyId`), based on the company's subscription plan:

Plan	Limit
STARTER	100 req / min
GROWTH	300 req / min
ENTERPRISE	1000 req / min

When the limit is exceeded:

- HTTP status: **429**
- Body: { "code": "RATE_LIMITED", "retryAfter": <seconds> }
- Frontend should implement retry with back-off using the `retryAfter` value.
- Display a user-friendly message: *"You've reached your plan's request limit. Please wait seconds or upgrade your plan."*

3. Idempotency (Mutation Safety)

For any `POST`, `PUT`, or `PATCH` request that should be executed exactly once (e.g., creating a revenue record, triggering AI analysis, executing an agent proposal), the frontend may optionally send an `X-Idempotency-Key` header.

How to use:

```
POST /api/v1/agent/proposals/:id/execute
Headers:
  Authorization: Bearer <token>
  X-Idempotency-Key: <uuid-v4> ← unique per logical operation
```

Behaviour:

- First request: executes normally, caches the response for 24 hours.
- Repeat request with same key: returns the **cached response** immediately without re-executing.
- After 24 hours: key expires; a new request with the same key executes fresh.

When to use idempotency keys:

- Executing an agent proposal (prevent double-execution on network retry)
- Creating revenue records (prevent duplicate entries on retry)
- Running AI analysis (prevent double billing quota consumption)

Generate a key: `crypto.randomUUID()` in modern browsers / Node.js.

4. Double Approval Workflow (Agent Actions)

When the AI Policy Engine determines an action is high-risk, it requires **double approval** from two different people before execution.

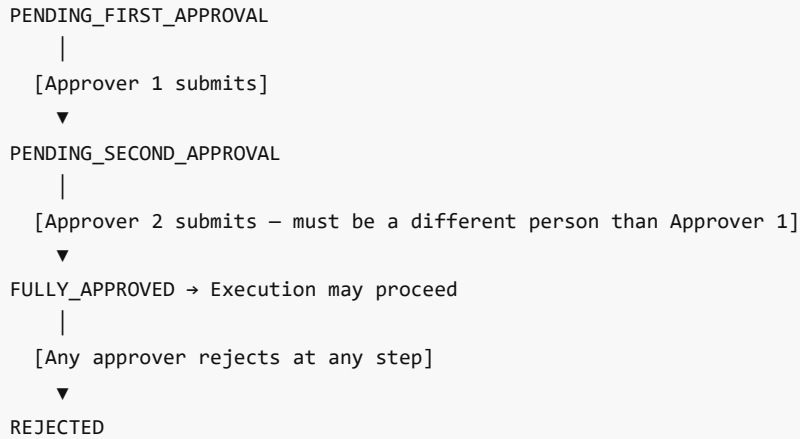
Trigger: `POST /agent/proposals/:id/execute` returns HTTP **202** with body:

```
{
  "success": true,
  "data": {
    "status": "APPROVAL_REQUIRED",
    "approvalId": "cmmXXX...",
  }
}
```



```
    "message": "This action requires double approval"
  }
}
```

Approval State Machine:



Frontend journey:

1. User clicks "Execute" on an approved proposal.
2. If response is HTTP 202 with `status: APPROVAL_REQUIRED` → show "Approval Required" banner.
3. Display approval status fetched from `GET /api/v1/agent/approvals/:approvalId`.
4. First approver clicks "Approve" (must be a `COMPANY_ADMIN`).
5. Status transitions to `PENDING_SECOND_APPROVAL` → notify a second admin.
6. Second approver (different from first) clicks "Approve".
7. Status becomes `FULLY_APPROVED` → call execute again to complete.

5. AI Policy Engine Verdicts

Before an agent action executes, the backend evaluates it against 8 policy rules. The verdict is returned in the execution response:

Verdict	HTTP Status	Frontend Action
ALLOW	200	Show success; action executed
REQUIRE_APPROVAL	202	Show approval workflow UI (see above)
DENY	403	Show error: <i>"This action is blocked by policy"</i>

DENY response shape:

```
{
  "success": false,
  "error": "Action blocked by policy rule: no-bulk-delete",
  "code": "POLICY_DENY",
  "data": {
    "rule": "no-bulk-delete",
    "verdict": "DENY"
  }
}
```

```
}  
}
```

6. AI Safety Verdicts

Every AI analysis result is safety-checked before being returned. The `AIInsight` and AI analysis responses may include a `safetyVerdict` field:

Verdict	Meaning	Frontend Action
SAFE	AI output passed all safety checks	Display normally
WARNING	Possible hallucination or uncertainty flagged	Show advisory banner: "Review AI output carefully"
BLOCKED	AI output blocked — contained injection or policy violation	Show error: "AI response blocked for safety"

Acceptance Criteria (QA Checklist)

Authentication

- ☐ `POST /api/v1/auth/login` returns `200` with `accessToken` and `refreshToken` for valid credentials
- ☐ Invalid credentials return `401`
- ☐ `POST /api/v1/auth/refresh` returns a new `accessToken`
- ☐ After login, an `AuditLog` row with `action = "LOGIN"` is created

Revenue

- ☐ `GET /api/v1/revenue` returns paginated records scoped to callers `companyId`
- ☐ `ANALYST` cannot `POST` or `DELETE` revenue records (403)
- ☐ `DELETE /api/v1/revenue/:id` creates a `DELETE_REVENUE` audit event

Leakages

- ☐ `PATCH .../resolve` sets `isResolved = true`, sets `resolvedAt`, creates `RESOLVE_LEAKAGE` audit event
- ☐ `VIEWER` receives `403` on resolve attempt

AI Engine

- ☐ `POST /api/v1/ai/run` returns a saved `AIInsight` with `severity`, `riskScore`, `title`
- ☐ `GET /api/v1/ai/insights` returns insights sorted by `createdAt DESC`
- ☐ `GET /api/v1/ai/usage` returns `totalRuns`, `quotaRemaining`
- ☐ `ANALYST` can read insights but cannot call `POST /ai/run`

AI Agent

- ☐ `POST /api/v1/agent/plan` with a valid `insightId` returns proposals with `totalCreated >= 1` on first call

- ☐ Second call with same `insightId` returns `totalCreated: 0`, `existingCount > 0` (dedup working)
- ☐ `POST .../approve` changes status to `APPROVED`
- ☐ `POST .../execute` on a `PROPOSED` (not approved) proposal returns `400`
- ☐ `POST .../execute` on `APPROVED` with `estimatedImpact < 100` is blocked
- ☐ Execution creates an `AgentExecutionLog` row
- ☐ `POST .../cleanup` removes duplicates and returns `{ deleted, message }`

Audit Trail

- ☐ `GET /api/v1/audit` returns `200` for `COMPANY_ADMIN`
- ☐ `GET /api/v1/audit` returns `403` for `ANALYST` and `VIEWER`
- ☐ `action=LOGIN` filter returns only login events
- ☐ `search=<email>` filters by actor email

Enterprise Governance (NEW v3.0)

- ☐ Every API response contains `requestId` (UUID-v4) and `timestamp` (ISO 8601) fields
- ☐ Every API response includes `X-Request-Id` header matching `requestId` in body
- ☐ `POST` request with `X-Idempotency-Key` replays cached response on repeat call within 24h
- ☐ Repeat idempotency key request does **not** execute the handler again (verify via DB — no duplicate row)
- ☐ `STARTER` plan company receives `429 RATE_LIMITED` after exceeding 100 requests/minute
- ☐ Rate limit response body includes `retryAfter` field (seconds until limit resets)
- ☐ High-risk agent execution returns HTTP `202` with `APPROVAL_REQUIRED` status
- ☐ `ActionApproval` record is created when double-approval is triggered
- ☐ Same approver cannot submit both approval steps (returns `400`)
- ☐ After two distinct approvals, `ActionApproval.status = FULLY_APPROVED`
- ☐ `AIPolicyEngine DENY` verdict returns HTTP `403` with `code: POLICY_DENY`
- ☐ AI response with safety violation returns `safetyVerdict: BLOCKED` or `WARNING`
- ☐ `AISafetyLog` row is created for every AI run (check DB directly)
- ☐ Failed connector events are retried up to 3 times then appear in `DeadLetterEvent` table
- ☐ `AgentExecutionLog` and `JobLog` rows created for executed jobs

General

- ☐ `/health` returns `{ status: "ok" }` when DB is reachable
- ☐ All POST endpoints with no body return `400` or `500` — send `{}` minimum
- ☐ Tenant isolation: users never see data from other companies

Test Cases

- Login with seeded `admin@demo.com` credentials returns `200` and `accessToken`.
- `GET /api/v1/audit` with the access token returns `{ success: true, data: { data: [...], meta: { total, page, ... } } }`.
- `GET /api/v1/audit?action=LOGIN` returns only LOGIN events.
- `GET /api/v1/audit?search=admin` returns entries where `actorEmail` or `actorName` contains "admin".
- List revenue for demo company returns non-empty list after seeding.
- List leakages returns items with `riskScore` and `category` fields.

- Run data migration runner twice — second run should skip previously executed migrations (`001_backfill_audit_logs`).

Operational Notes for Product

- Demo data is for POC/testing only. Rotate or remove demo credentials before public release.
- Use the `DataMigrationHistory` table as the audit-log for non-schema data migrations.
- Audit logs are tenant-scoped — a `COMPANY_ADMIN` can only see their own company's events.
- Actor fields (`actorEmail` , `actorName`) are stored at write-time; they will not reflect future user profile changes.
- All audit writes are fire-and-forget — they will not cause a request to fail if the DB write fails.
- To expand audit coverage, call `createAuditLog(request.server.prisma, { ... })` in any controller method that performs a significant action.

Contact: Product / QA team